

Pete | Artist — Portfolio Website Redesign

Project Description

This project is a redesign of a single-page portfolio website for "Pete," a fictional artist based in Denver, Colorado. The site is built with plain HTML and CSS (no frameworks or build tools) and includes a header/navigation bar, an About section, a Portfolio section showcasing three projects, and a Contact section, capped off with a footer.

The redesign work focused on three areas of the page: the header, the About section, and the Portfolio section. In each case, the goal was to take an existing layout and restructure it to meet a new visual spec — primarily using CSS Flexbox to control alignment, spacing, and row/column layout — while keeping the color palette, typography, and overall "warm, editorial" theme of the original design intact.

Original vs. Revised Layout

Header

- **Original:** The logo (`<h1>Pete | Artist</h1>`) and navigation links sat as separate block-level elements, stacking vertically instead of forming a single row. The header also had no defined width, so it stretched edge-to-edge rather than aligning with the rest of the page's content column.
- **Revised:** `header` was set to `display: flex` with `justify-content: space-between` and `align-items: center`, placing the logo on the left and navigation links on the right, vertically centered on one row. A `max-width` plus `margin: 0 auto` was added to center the header horizontally, matching the centering pattern (`margin: auto`) already used elsewhere on the page.

About Section

- **Original:** The `<h3>Hi! I'm Pete</h3>` heading, the profile image, and the supporting paragraphs/list were all direct siblings inside one `<section>`, causing everything to stack vertically in one column.
- **Revised:** The image and the text content were split into two wrapper `<div>`s (`.intro-image` and `.intro-text`) inside a new `.intro` container. `.intro` uses `display: flex` to place the image and text side by side on one row. The heading (`.intro-text h3`) stays centered above the text, while the paragraphs and list are

left-aligned for readability. The "Pete's Background" section remains a separate sibling `<section>` below, unaffected by the row layout above it.

Portfolio Section

- **Original:** The three portfolio `<section>` elements ("Abstract Red," "Spiral Zany," "Melted Rainbow") were direct children of `<article id="portfolio">`, stacking vertically one after another.
- **Revised:** The three sections were wrapped in a new `<div class="portfolio-section">`, which uses `display: flex` and `justify-content: space-between` to lay the three projects out in a single row with consistent spacing. Each project's image was resized down from the site-wide 200px cap to 160px so all three columns fit within the page's existing 600px content width, and the description paragraphs were set to the required 14px font size and centered beneath each image.

Redesign Mock-ups

Each section was redesigned against a provided mock-up image showing the target layout:

- **Header mock-up:** logo and nav links on a single centered row.
- **About mock-up:** profile photo and intro text side by side, heading centered, body text left-aligned, background section below.
- **Portfolio mock-up:** three projects in a single row with even spacing and small, consistent caption text.

Implementation Plan

1. Identify the structural change needed to satisfy each mock-up (e.g., "these elements need to be flex siblings" or "these need a shared wrapper").
2. Add or adjust minimal HTML wrapper elements (`<div>` containers with descriptive class names) only where the existing markup didn't already support the needed grouping — no unrelated markup was changed.
3. Apply Flexbox rules scoped to new class selectors (`.intro`, `.portfolio-section`, etc.) rather than editing broad element selectors like `div` or `section`, to avoid breaking layout elsewhere on the page.
4. Reuse the existing color palette, fonts, and spacing patterns already established in `style.css` so each redesigned section still feels like part of the same site.
5. Verify each change didn't conflict with existing global rules (e.g., the site-wide `div { width: 600px; margin: auto; }` and `article div { text-align:`

`center; width: 100%; }` rules), adjusting specificity with classes where an override was needed.

Design Trade-offs

- **Class-based overrides vs. rewriting global rules:** The project's CSS leans on broad element selectors (`div`, `article div`, `img`) to keep the file short. Rather than rewriting these global rules — which risked breaking sections not being redesigned — new, more specific class selectors (e.g., `.intro-text`, `.portfolio-section img`) were added to override just the properties that needed to change. This keeps the original CSS structure intact but means the stylesheet now mixes both approaches (global element rules and targeted class rules), which a larger project would probably want to consolidate.
- **Fixed image sizing vs. fluid sizing:** Portfolio images were given a fixed `max-width` (160px) tuned to fit three columns inside the existing 600px content width, rather than a fully fluid/percentage-based approach. This keeps the row from wrapping at the current content width, but means the sizing would need to be revisited if the site's overall content width or number of portfolio items changes.
- **flex-start vs. center alignment in the About row:** The image and text column in the About section are aligned with `align-items: flex-start` so the photo sits at the top of the (taller) text block, rather than vertically centering the two columns against each other. This was a stylistic choice to match the mock-up; centering was considered but looked visually unbalanced given the text column's height.

AI Tools Used

Claude (Anthropic) was used throughout this project as a coding assistant, via the Claude.ai chat interface.

Justification for use:

- README creation.
- Diagnosing layout bugs by reading the existing HTML/CSS together and identifying which rule (or missing wrapper element) was causing a given section to render incorrectly (e.g., the portfolio images stacking instead of forming a row).
- Explaining *why* each CSS change works (specificity, the flexbox model, inheritance) so the changes could be understood and defended.

All final HTML and CSS was reviewed against the assignment's mock-up images before being accepted into the project.

Key Decisions, Challenges, and Learning Moments

- **Decision — scoping new CSS to classes:** Early on, it became clear that the project's CSS relies heavily on generic element selectors (`div`, `img`, `h1`, `h2`, `h3`). Any new layout change had to be added as a class-based rule with higher specificity, rather than editing those global rules directly, to avoid unintended side effects on other sections of the page.
- **Challenge — header not centering:** After adding `max-width` to the `header` rule, the header was still flush against the left edge of the page instead of centered. The issue was a missing `margin: 0 auto` — `max-width` alone only caps a box's width, it doesn't tell the browser where to position the leftover space. This was a useful reminder that width constraints and centering are two separate CSS concerns.
- **Challenge — About section not matching the mock-up:** Initial attempts to redesign the About section were based on a misread of which screenshot represented the "before" state versus the target "after" state, leading to an incorrect first pass (a stacked layout with a muted heading color). Once the actual written spec was provided, the section was corrected to the intended side-by-side layout. This was a good lesson in confirming the actual written requirements rather than inferring intent purely from screenshots.
- **Challenge/Debugging — portfolio images stacking instead of forming a row:** After adding the `.portfolio-section { display: flex; }` rule, the three portfolio projects were still stacking vertically. The root cause was that the CSS rule alone does nothing without the matching HTML wrapper (`<div class="portfolio-section">`) actually present around the three `<section>` elements in the markup — a reminder that a CSS rule targeting a class only takes effect once that class is correctly applied in the HTML.
- **Learning moment — specificity in practice:** Working within an existing stylesheet built on element selectors was a practical demonstration of CSS specificity rules: a class selector (`.intro-text`) will always override a two-element selector (`article div`) regardless of the order the rules appear in the file, which made it possible to make targeted changes without touching the original rules at all.